

QA Test Credentials

Support Ticketing System — seeded demo accounts for functional and security testing

These accounts exist and were verified working

Every account below was created by the development seeder and signed in successfully against a live SQL Server database on 18 August 2026. The permission counts are the actual numbers returned by the API, not estimates.

Scope: Phase 1 delivers authentication, organizations, users, roles, permissions, teams and master data. Ticket creation, SLA, escalation, dashboards and the React frontend are **not built yet**, so the only endpoints available are those listed on page 3.

Before you use these

- The shared password is **weak by design** and printed in a document. It is safe only on an isolated development machine.
- The seeder refuses to run unless **both** `ASPNETCORE_ENVIRONMENT=Development` and `Seed:EnableDemoAccounts=true` are set, and it aborts if the database already contains an organization.
- No password is compiled into the codebase. The value comes from `Seed:DemoPassword` in `user-secrets`. If that is not set the seeder generates a random one and prints it to the console once.
- The `.test` TLD is reserved by RFC 6761, so no message can reach a real mailbox.
- Treat this document as internal. Do not attach it to a public ticket, wiki or repository.

Environment

Item	Value	Notes
API	<code>http://localhost:5180/api/v1</code>	ASP.NET Core 10
Swagger	<code>http://localhost:5180/swagger</code>	Development only
Health	<code>/health/live</code> and <code>/health/ready</code>	Anonymous; ready checks the database
Database	<code>Server=.</code> ; <code>Database=SupportTicketing</code>	SQL Server 2025, Windows auth
Password	<code>Seed:DemoPassword</code>	Shared by every account below
Tenants	ITG Group / Fabrikam Trading	Two organizations for isolation testing

Organization A — ITG Group

The primary tenant. Departments: Information Technology and Operations. Teams: IT Support and ERP Support. Ten accounts covering all seven roles.

Seeded accounts					
Role	Email	Name	Perms	Dept / Team	What it is for
Requester	requester@itg.test	Rabia Khan	10	Operations	Primary requester. Raises tickets, confirms resolution, submits CSAT.
Requester	requester2@itg.test	Omar Siddiqui	10	Operations	Second requester. Must not see requester1's tickets.
Support Agent	agent@itg.test	Ayesha Malik	23	IT / IT Support	Primary assignee. Accept, work, internal notes, resolve.
Support Agent	agent2@itg.test	Bilal Ahmed	23	IT / IT Support	Second agent for round-robin and reassignment.
Support Agent	erpagent@itg.test	Sana Iqbal	23	IT / ERP Support	Cross-team transfer target.
Team Lead	lead@itg.test	Imran Sheikh	35	IT / IT Support	Assign, reassign, change priority, close, escalate.
Technical Specialist	specialist@itg.test	Zainab Raza	25	IT / ERP Support	Escalated technical work, root cause, transfer.
Manager	manager@itg.test	Faisal Qureshi	42	IT	Cross-team dashboards, SLA override, exports.
Administrator	admin@itg.test	Nadia Hussain	17	IT	Users, roles, catalogue, SLA config. No blanket ticket access.
Super Admin	superadmin@itg.test	Kamran Ali	55	IT	Full control. Every permission in the catalogue.

Organization B — Fabrikam Trading

A second tenant so isolation is proven rather than assumed. Every cross-tenant read must fail, and it must fail as **404 Not Found** — a 403 confirms the record exists.

Role	Email	Name	Perms	What it is for
Requester	requester@fab.test	Emma Clarke	10	Cross-tenant probe. Must never read an ITG ticket, file or user.
Support Agent	agent@fab.test	Daniel Reid	23	Must not appear in ITG assignee lists.
Administrator	admin@fab.test	Sophie Turner	17	Must not read or modify ITG configuration.

Two deliberate traps

- **Administrator holds 17 permissions and none of them is `ticket.view_all`.** Managing users and configuration does not imply a right to read support conversations. If an administrator can read an arbitrary ticket body once ticketing ships, raise a defect.
- **Requester holds 10 permissions and scope `Own`.** If a requester can see another requester's ticket, raise a defect.

Endpoints available in Phase 1

These are the only endpoints that exist today. Everything else returns 404 because it has not been built yet, not because of a defect.

Method	Path	Auth	Notes
POST	/api/v1/auth/login	Anonymous	Sign in. Rate limited to 10 per minute per address.
POST	/api/v1/auth/refresh	Anonymous	Rotate the refresh token. Single use; replay kills the family.
POST	/api/v1/auth/logout	Bearer	Revoke the current session, or all sessions.
GET	/api/v1/auth/me	Bearer	Profile, roles and effective permissions.
GET	/health/live	Anonymous	Liveness. No dependency checks.
GET	/health/ready	Anonymous	Readiness. Verifies the database connection.
GET	/swagger	Anonymous	Interactive API documentation, Development only.

Security behaviour already verified

Each row was executed against the running API and is covered by an automated test. The suite is 46 tests: 18 unit, 7 architecture and 21 integration, all passing.

Behaviour	Result	Evidence
Sign-in issues a token carrying the user's permissions	Pass	23 permission claims for a Support Agent
Data scope is derived from the role, not the request	Pass	scp=3 (Team) for an agent, 6 (All) for Super Admin
Administrator does not hold ticket.view_all	Pass	Asserted in the integration suite
Refresh rotates the token and reissues permissions	Pass	New refresh token differs from the old one
Replaying a rotated refresh token kills the family	Pass	Returns 401 refresh_token_reused; sibling tokens die too
Unknown email and wrong password are indistinguishable	Pass	Identical status and detail, matched hashing cost
Failed sign-ins are written to the audit trail	Pass	Row present, attempted password absent from it
Lockout counter survives the failed-login exception	Pass	AccessFailedCount reaches 2 after two failures
Sign-in is rate limited per client address	Pass	11th attempt in a minute returns 429
Two organizations resolve to separate tenants	Pass	Distinct org claim and organization name
Errors never leak a stack trace or SQL	Pass	Problem Details carries only a correlation id
Unauthenticated and malformed tokens are rejected	Pass	401 on /auth/me with no or invalid bearer token

How to run it

- Set the two secrets, from the repository root:

```
dotnet user-secrets set "Jwt:SigningKey" "<48+ random chars>" --project src/SupportTicketing.Api
dotnet user-secrets set "Seed:DemoPassword" "<your password>" --project src/SupportTicketing.Api
```
- Apply the migrations:

```
dotnet ef database update --project src/SupportTicketing.Infrastructure --startup-project
src/SupportTicketing.Api
```
- Start the API:

```
dotnet run --project src/SupportTicketing.Api
```

The seeder runs at startup and logs how many permissions and organizations it created.
- Sign in at /swagger, or post to /api/v1/auth/login, then paste the access token into Swagger's Authorize dialog.